

Experiment 1

Aim: Write a program to implement inter process communication

Theory:

Interprocess Communication

Interprocess Communication in a distributed system is a process of exchanging data between two or more independent processes in a distributed environment called Interprocess communication. Interprocess communication on the internet provides both Datagram and stream communication.

Characteristics of Inter-process Communication in Distributed Systems

- **Synchronous System Calls:** In synchronous system calls both sender and receiver use blocking system calls to transmit the data which means the sender will wait until the acknowledgment is received from the receiver and the receiver waits until the message arrives.
- **Asynchronous System Calls:** In asynchronous system calls, both sender and receiver use non-blocking system calls to transmit the data which means the sender doesn't wait from the receiver acknowledgment.
- **Message Destination:** A local port is a message destination within a computer, specified as an integer. A port has exactly one receiver but many senders. Processes may use multiple ports from which to receive messages. Any process that knows the number of a port can send the message to it.
- **Reliability:** It is defined as validity and integrity.
- **Integrity:** Messages must arrive without corruption and duplication to the destination.

Types of Interprocess Communication in Distributed Systems

Message Passing:

- **Definition:** Message passing involves processes communicating by sending and receiving messages. Messages can be structured data packets containing information or commands.
- **Characteristics:** It is a versatile method suitable for both synchronous and asynchronous communication. Message passing can be implemented using various protocols such as TCP/IP, UDP, or higher-level messaging protocols like

AMQP (Advanced Message Queuing Protocol) or MQTT (Message Queuing Telemetry Transport).

Remote Procedure Calls (RPC):

- Definition: RPC allows one process to invoke a procedure (or function) in another process, typically located on a different machine over a network.
- Characteristics: It abstracts the communication between processes by making it appear as if a local procedure call is being made. RPC frameworks handle details like parameter marshalling, network communication, and error handling.

Sockets:

- Definition: Sockets provide a low-level interface for network communication between processes running on different computers.
- Characteristics: They allow processes to establish connections, send data streams (TCP) or datagrams (UDP), and receive responses. Sockets are fundamental for implementing higher-level communication protocols.

Message Queuing Systems:

- Description: Message queuing systems facilitate asynchronous communication by allowing processes to send messages to and receive messages from queues.
- Characteristics: They decouple producers (senders) and consumers (receivers) of messages, providing fault tolerance, scalability, and persistence of messages. Examples include Apache Kafka, RabbitMQ, and AWS SQS.

Publish-Subscribe Systems:

- Description: Publish-subscribe (pub-sub) systems enable communication between components without requiring them to directly know each other.
- Characteristics: Publishers publish messages to topics, and subscribers receive messages based on their interest in specific topics. This model supports one-to-many communication and is scalable for large-scale distributed systems. Examples include MQTT and Apache Pulsar.

These types of IPC mechanisms each have distinct advantages and are chosen based on factors such as communication requirements, performance considerations, and the nature of the distributed system architecture. Successful implementation often involves selecting the most suitable IPC type or combination thereof to meet specific application needs.

Code:

1. Server.java

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main(String[] args) {
        int PORT = 4568;

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server running on port " + PORT + "...
Waiting for client...");

            try (Socket socket = serverSocket.accept();
                BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                PrintWriter output = new
PrintWriter(socket.getOutputStream(), true)) {

                System.out.println("Client connected: " +
socket.getInetAddress());

                // Read two numbers
                int a = Integer.parseInt(input.readLine());
                int b = Integer.parseInt(input.readLine());

                System.out.println("Received numbers: " + a + ", " + b);

                // Send result
                output.println(a + b);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2. Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

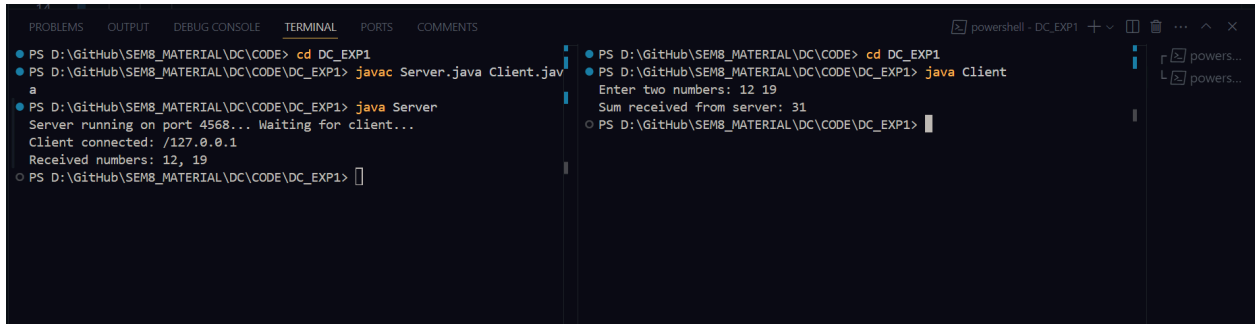
public class Client {
    public static void main(String[] args) {
        String HOST = "127.0.0.1";
        int PORT = 4568;

        try {
            Socket socket = new Socket(HOST, PORT);
            PrintWriter output = new
PrintWriter(socket.getOutputStream(), true);
            BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            Scanner scanner = new Scanner(System.in) {

                System.out.print("Enter two numbers: ");
                output.println(scanner.nextInt());
                output.println(scanner.nextInt());

                System.out.println("Sum received from server: " +
input.readLine());
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS
PS D:\Github\SEM8_MATERIAL\DC\CODE> cd DC_EXP1
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP1> javac Server.java Client.java
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP1> java Server
Server running on port 4568... Waiting for client...
Client connected: /127.0.0.1
Received numbers: 12, 19
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP1>

PS D:\Github\SEM8_MATERIAL\DC\CODE> cd DC_EXP1
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP1> java client
Enter two numbers: 12 19
Sum received from server: 31
PS D:\Github\SEM8_MATERIAL\DC\CODE\DC_EXP1>
```